

Longest Possible Route in a Matrix with Hurdles (Backtracking + DFS)

Problem Statement

Given a binary matrix:

- 1 → Free cell
- 0 → Blocked cell

Find the **longest possible path** from (xs, ys) to (xd, yd) .

Rules:

- Move only Up, Down, Left, Right.
- A cell can be visited **only once**.
- Return -1 if destination cannot be reached.

Example

```
S . . .  
. X . D  
. . . .
```

Output:

```
Longest path length
```

What are they actually asking?

They are **NOT** asking

- Shortest Path ❌
- Number of Paths ❌
- Whether a Path Exists ❌

Instead,

Among every valid path from source to destination, return the one having the **maximum length**.

That one sentence changes the entire approach.

First Thought Process (How to Build the Logic)

Whenever you read

Longest Path

ask yourself

Can I greedily choose a direction?

No.

Example

S → D

Length = 2

Another path

S
↓

↓

→→→

↑

D

Length = 10

The first path reached destination quickly.

The second path is much longer.

So stopping at the first answer is incorrect.

We must explore **every possible path**.

Step 1 : Which Algorithm?

Need to explore all paths

↓

DFS

Need to avoid revisiting

↓

Visited Array

Need to explore every possibility

↓

Backtracking

So

DFS + Backtracking

Why not BFS?

BFS naturally finds

Shortest Path

because it explores level by level.

Longest simple path has no greedy property.

Every path must be checked.

Main Observation

Every cell has multiple choices.

Cell
/ | | \
U D L R

Each direction can produce a different path length.

Therefore

Current Answer

=

Maximum

(

Up,
Down,
Left,
Right

)

Recursive Thinking

Suppose currently at

(r, c)

Question becomes

"What is the longest path from THIS cell to destination?"

Notice

We are solving the **same problem** repeatedly.

Perfect recursion.

State of DFS

Every recursive call only needs

Current Row

Current Column

Visited Cells

Nothing else.

Base Cases

1. Outside Grid

```
return -∞
```

Invalid path.

2. Blocked Cell

```
return -∞
```

Cannot walk there.

3. Already Visited

```
return -∞
```

Cannot revisit.

4. Destination Reached

```
return 0
```

This is the most confusing part.

Let's understand carefully.

Why Return 0 at Destination?

Suppose

```
A → B → C(Destination)
```

When recursion reaches

```
C
```

Question is

```
How many more moves are needed from C to reach C?
```

Answer

```
0
```

No movement required.

So

```
return 0
```

Why do we Return best + 1 ?

This is the most important concept.

Suppose

```
A → B → C → D
```

Destination = D

At D

```
return 0
```

Meaning

Distance from D to D

=

0

At C

Recursive call says

Longest distance from D

=

0

But

C must move

C → D

That is

1 move

So

return 0 + 1

=

1

At B

Recursive answer

Longest distance from C

=

1

Need one more move

B → C

Therefore

return 1 + 1

=

2

At A

Recursive answer

2

Need one more move

A → B

Return

3

Complete table

Current Cell	Child Returns	Current Returns
D	0	0
C	0	1
B	1	2
A	2	3

Notice

Every parent contributes

One extra edge

+

Child Answer

Hence

```
return best + 1
```

Why not return best?

Suppose

$A \rightarrow B \rightarrow D$

Destination returns

0

If B also returns

0

Then A returns

0

Entire path length becomes

0

✖ Wrong

Every parent must count the move taken to reach its child.

Hence

Child Answer + Current Edge

=

best + 1

Backtracking

Before exploring

Mark Visited

After exploring

Unmark

`vis[r][c]=True`

Explore

`vis[r][c]=False`

Without unmarking

S

/ \

A B

After exploring

S→A

A stays visited.

Now

S→B→A

becomes impossible.

Which is wrong.

Every path should get a fresh chance.

Building the Code

Step 1

Boundary

```
if r<0 or r>=n or c<0 or c>=m:  
    return float("-inf")
```

Step 2

Blocked

```
if mat[r][c]==0:  
    return float("-inf")
```

Step 3

Visited

```
if vis[r][c]:  
    return float("-inf")
```

Step 4

Destination

```
if r==xd and c==yd:  
    return 0
```

Step 5

Mark

```
vis[r][c]=True
```

Step 6

Explore all four directions

```
best=max(  
    dfs(up),  
    dfs(down),  
    dfs(left),  
    dfs(right)  
)
```

Step 7

Undo

```
vis[r][c]=False
```

Step 8

No valid path

```
if best==float("-inf"):  
    return float("-inf")
```

Step 9

Otherwise

```
return best+1
```

Dry Run

```
S . D
```

Call

```
dfs(S)
```

↓

Moves Right

↓

```
dfs(.)
```

↓

Moves Right

↓

```
dfs(D)
```

Returns

```
0
```

Parent

$0+1$

=

1

Root

 $1+1$

=

2

Final Answer

2 moves

Complexity

Time

Exponential

 $O(4^{(N \times M)})$

because every simple path is explored.

Space

 $O(N \times M)$

Visited array + recursion stack.

Pattern Recognition Template

Whenever you see

Longest

Maximum

Minimum

All Possible Paths

Visit Once

No Revisit

Grid / Graph

Immediately think

DFS

+

Backtracking

Quick Recognition Checklist

- ✓ Need every possible path
- ✓ Cannot revisit nodes
- ✓ Maximum or Minimum answer
- ✓ Four-direction movement
- ✓ No greedy choice exists

↓

Use

DFS + Backtracking

Generic Backtracking Template

```
def dfs(state):  
    if invalid:  
        return  
  
    if answer:  
        return value  
  
    mark()  
  
    ans = combine(  
        dfs(choice1),  
        dfs(choice2),  
        dfs(choice3),  
        dfs(choice4)  
    )  
  
    unmark()  
  
    return ans
```

Similar Interview Problems

- Longest Path in Matrix
- Rat in a Maze (All Paths)
- Unique Paths III
- Word Search
- Hamiltonian Path
- Hamiltonian Cycle
- N Queens
- Sudoku Solver
- Knight's Tour
- All Paths from Source to Target

Mental Framework (30-second Interview Trick)

Read the problem.

↓

Ask

Can I stop after finding one answer?

Yes

↓

Normal DFS/BFS

No

↓

Need every possibility

↓

DFS

↓

Can a node be reused?

No

↓

Visited Array

↓

Need to undo after recursion?

Yes

↓

Backtracking

Pattern Learned Today

Longest Path + Visit Once

↓

Explore Every Possibility

↓

DFS

↓

Visited Array

↓

Backtracking

↓

Return `max(child answers) + 1`

One-Line Memory Rule

Whenever a problem asks for the longest (or maximum) simple path where nodes cannot be revisited, think "DFS + Backtracking": explore every possible path, undo the visited state after each branch, and return `best + 1` because every recursive parent contributes one additional move to the best path returned by its child.